What a Composition looks like

One render step creates every resource – but on its own it isn't enough

composition.yaml · MyApp, render step only

```yaml
apiVersion: apiextensions.crossplane.io/v1
kind: Composition
spec:
  compositeTypeRef:          # which XR it serves
    apiVersion: crossplane.best-practice.com/v1alpha1
    kind: MyApp
  mode: Pipeline
  pipeline:
  - step: render
    functionRef:
      name: function-go-templating
    input:                   # template emits the MRs
      kind: GoTemplate
      inline:
        template: …   # Bucket, Role, Policy, Attachment
```

THE CHALLENGES

1. **Wrong order**
   All MRs are applied at once – the attachment fails until Role and Policy exist.
2. **Values that don't exist yet**
   The Policy needs the bucket ARN, which AWS returns only after create.
3. **Never Ready**
   Nothing marks the composed resources Ready, so the XR stays not Ready.
4. **Hidden AWS errors**
   A failed update keeps the XR Ready – ArgoCD says Healthy.

Each challenge gets its own small function – next: the full MyApp pipeline.



What a Composition looks like

One render step creates every resource – but on its own it isn't enough

composition.yaml · MyApp, render step only

```yaml
apiVersion: apiextensions.crossplane.io/v1
kind: Composition
spec:
  compositeTypeRef:          # which XR it serves
    apiVersion: crossplane.best-practice.com/v1alpha1
    kind: MyApp
  mode: Pipeline
  pipeline:
  - step: render
    functionRef:
      name: function-go-templating
    input:                   # template emits the MRs
      kind: GoTemplate
      inline:
        template: …   # Bucket, Role, Policy, Attachment
```

THE CHALLENGES

1. **Wrong order**
   All MRs are applied at once – the attachment fails until Role and Policy exist.
2. **Values that don't exist yet**
   The Policy needs the bucket ARN, which AWS returns only after create.
3. **Never Ready**
   Nothing marks the composed resources Ready, so the XR stays not Ready.
4. **Hidden AWS errors**
   A failed update keeps the XR Ready – ArgoCD says Healthy.

Each challenge gets its own small function – next: the full MyApp pipeline.

What a Composition looks like

One render step creates every resource – but on its own it isn't enough

composition.yaml · MyApp, render step only

```yaml
apiVersion: apiextensions.crossplane.io/v1
kind: Composition
spec:
  compositeTypeRef:          # which XR it serves
    apiVersion: crossplane.best-practice.com/v1alpha1
    kind: MyApp
  mode: Pipeline
  pipeline:
  - step: render
    functionRef:
      name: function-go-templating
    input:                   # template emits the MRs
      kind: GoTemplate
      inline:
        template: …   # Bucket, Role, Policy, Attachment
```

THE CHALLENGES

1. **Wrong order**
   All MRs are applied at once – the attachment fails until Role and Policy exist.
2. **Values that don't exist yet**
   The Policy needs the bucket ARN, which AWS returns only after create.
3. **Never Ready**
   Nothing marks the composed resources Ready, so the XR stays not Ready.
4. **Hidden AWS errors**
   A failed update keeps the XR Ready – ArgoCD says Healthy.

Each challenge gets its own small function – next: the full MyApp pipeline.

THE CHALLENGES

1

Wrong order

All MRs are applied at once – the attachment fails until Role and Policy exist.

2

Values that don't exist yet

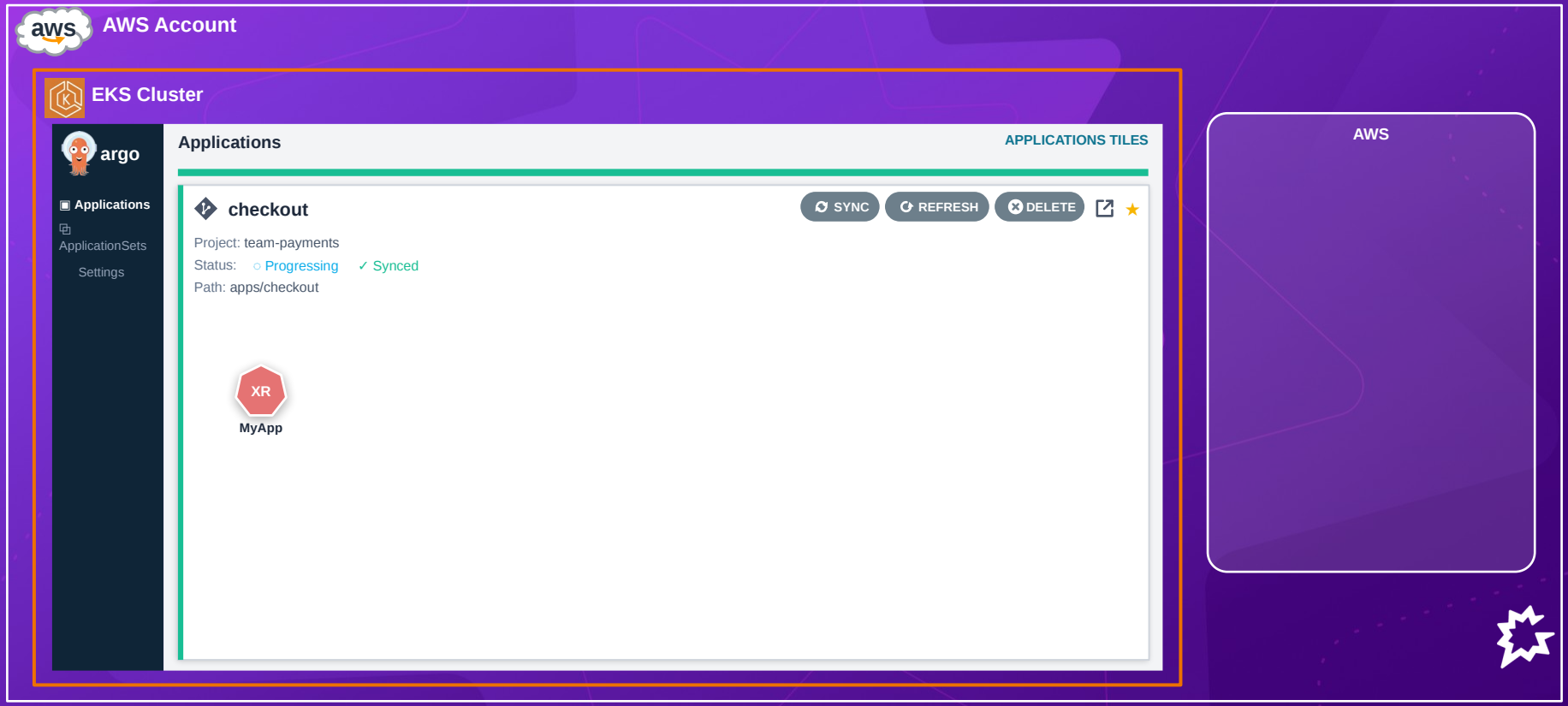
The Policy needs the bucket ARN, which AWS returns only after create.

Challenge 1 · Wrong order

All MRs are applied at once – the attachment fails until Role and Policy exist.

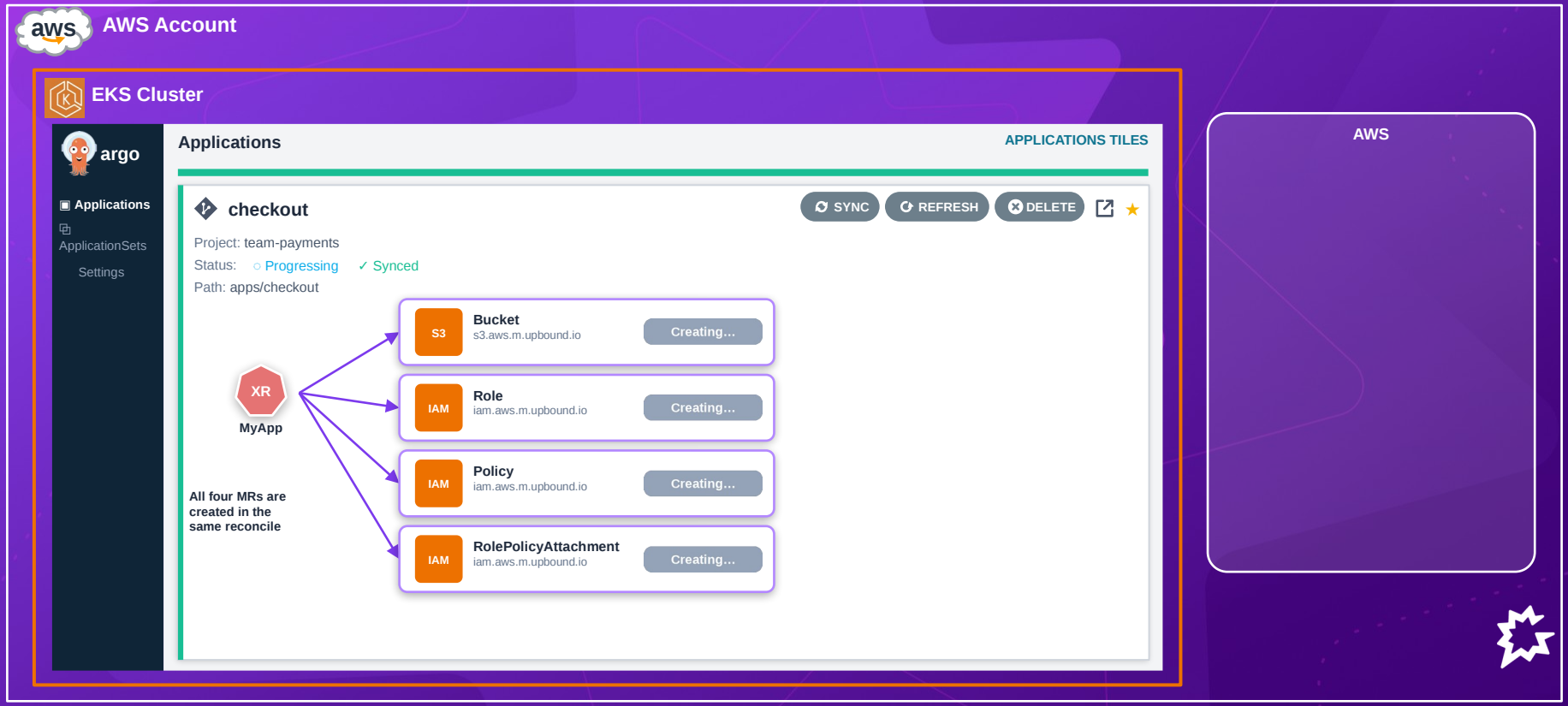
Challenge 1 · Wrong order

All MRs are applied at once – the attachment fails until Role and Policy exist.



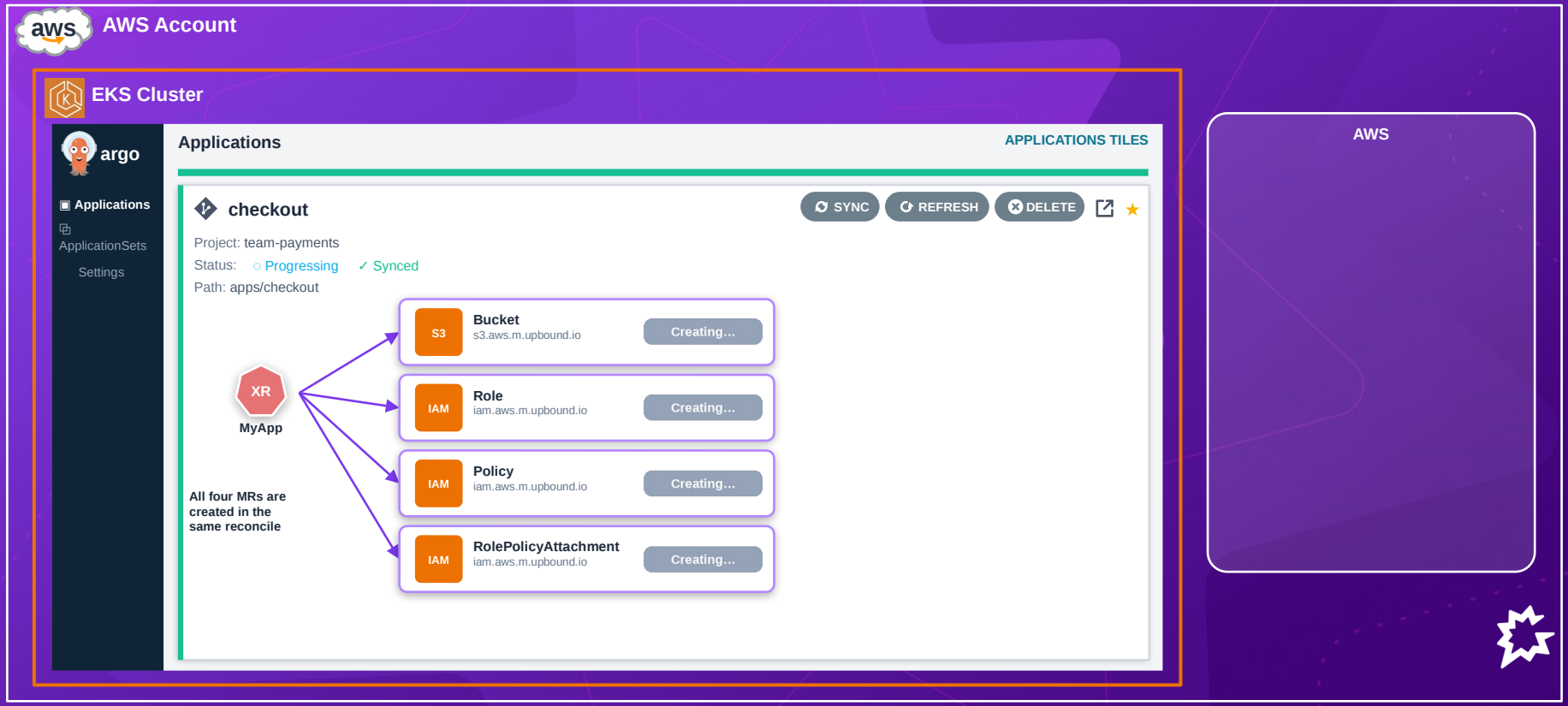
Challenge 1 · Wrong order

All MRs are applied at once – the attachment fails until Role and Policy exist.



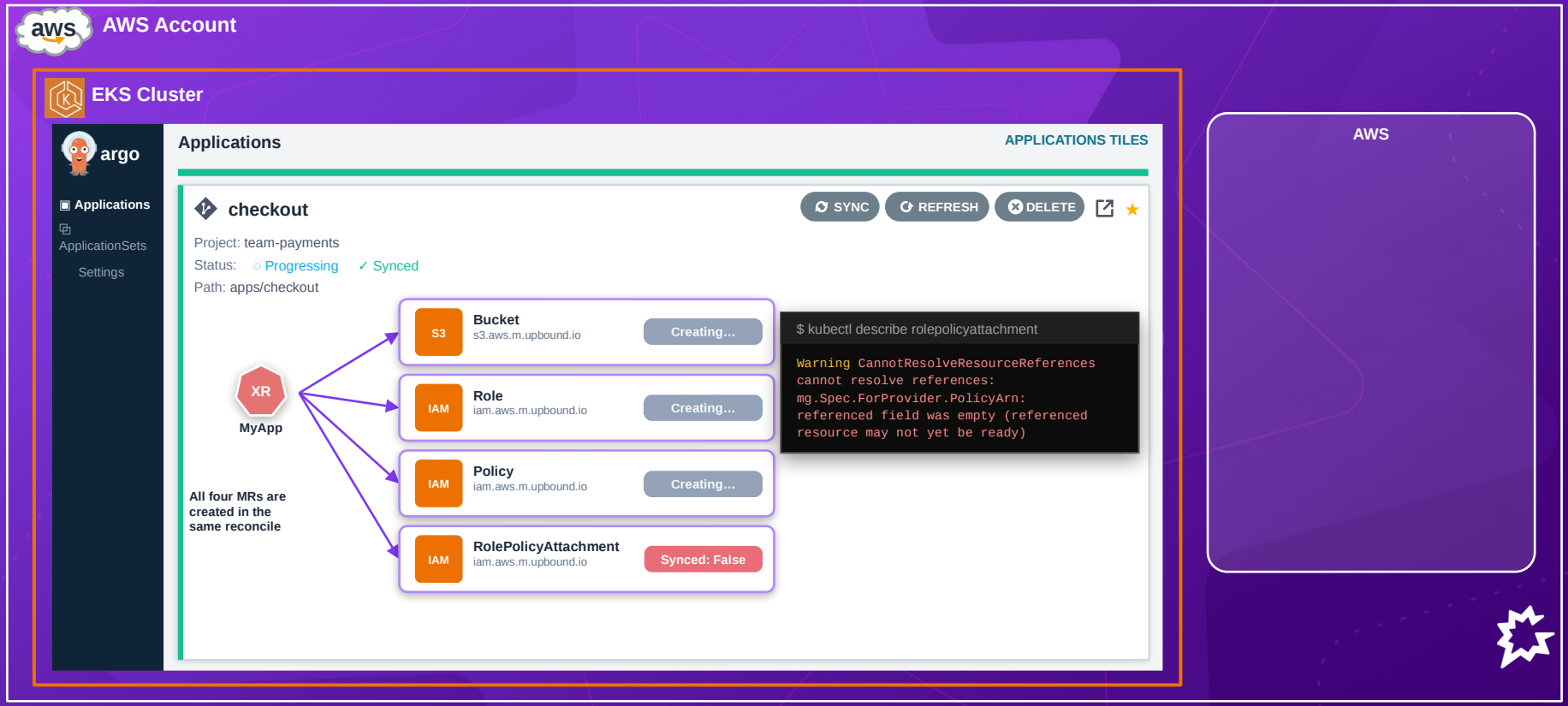
Challenge 1 · Wrong order

All MRs are applied at once – the attachment fails until Role and Policy exist.



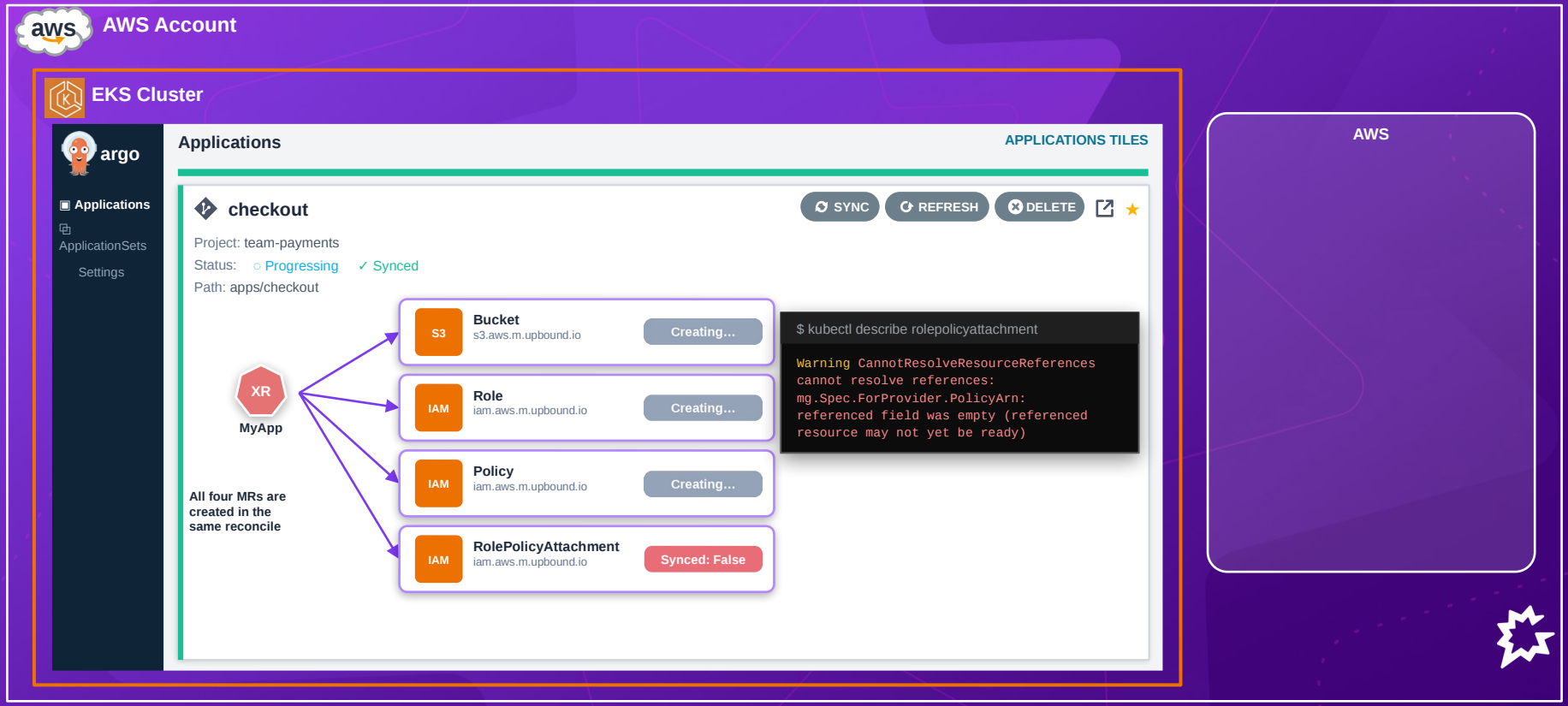
Challenge 1 · Wrong order

All MRs are applied at once – the attachment fails until Role and Policy exist.



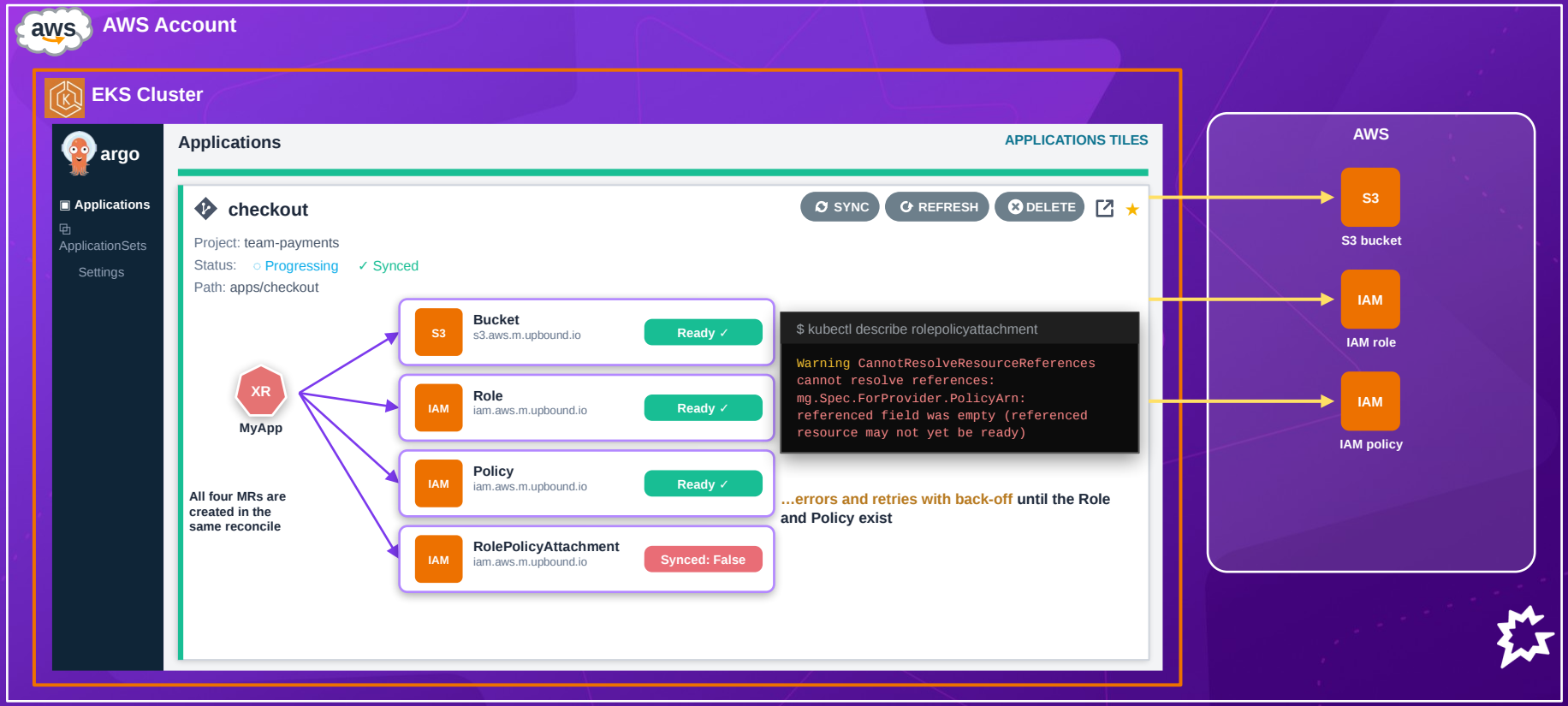
Challenge 1 · Wrong order

All MRs are applied at once – the attachment fails until Role and Policy exist.



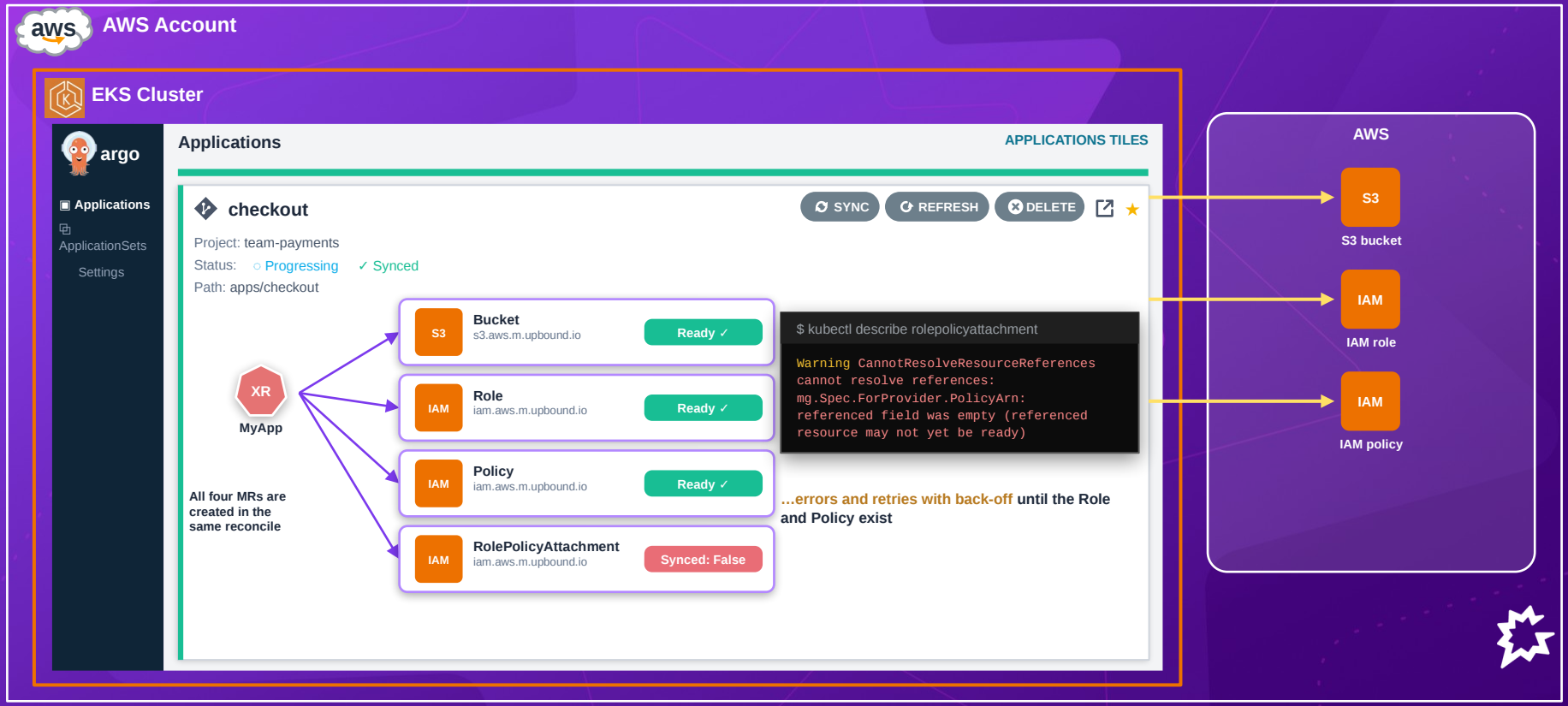
Challenge 1 · Wrong order

All MRs are applied at once – the attachment fails until Role and Policy exist.



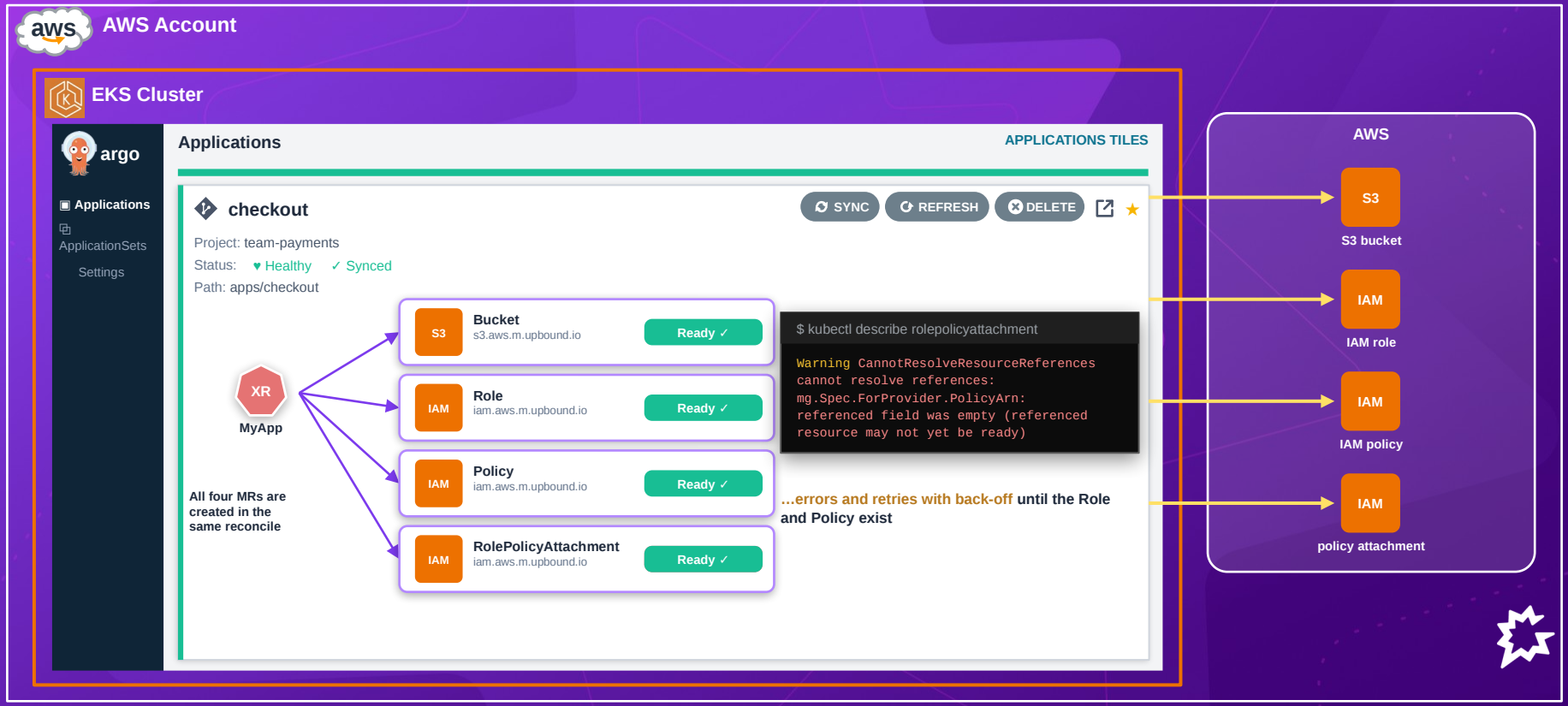
Challenge 1 · Wrong order

All MRs are applied at once – the attachment fails until Role and Policy exist.



Challenge 1 · Wrong order

All MRs are applied at once – the attachment fails until Role and Policy exist.



Challenge 1 · Wrong order

All MRs are applied at once – the attachment fails until Role and Policy exist.

